Glowy

Dossier P4 : Avis, modération et support

Projet d'équipe · octobre 2026

Ce dossier explique le projet en détail, puis **ta gestion** : ce qu'elle fait, ses règles métier, son IA, les fichiers à connaître et comment la présenter à la validation.

Le projet Glowy en détail

1. Qu'est-ce que Glowy ?

Glowy est une **plateforme de réservation de salons de beauté vérifiés en Tunisie** (coiffure, barbier, ongles, spa, visage…). Une cliente cherche un salon, voit les prestations, les prix en dinars, les avis et l'affluence, puis réserve un créneau précis en moins de 2 minutes. Le salon gère tout son travail depuis un tableau de bord, et un Super Admin contrôle la plateforme.

- **4 rôles** : cliente, employé, gérant de salon, Super Admin.
- **3 langues** : français, arabe (écriture de droite à gauche) et anglais.
- **Application installable** sur téléphone (PWA), utilisable hors connexion.
- **5 intelligences artificielles**, une par membre de l'équipe.
- Projet d'équipe de 5 étudiants : chaque personne a **1 gestion classique + 1 IA**.

2. Les technologies

Domaine	Technologie	Rôle dans le projet
Langage	**TypeScript**	partout : pages, API, règles métier, tests
Framework	**Next.js 16** (App Router) + **React 19**	site et API dans un seul projet
Interface	**Tailwind CSS 4**, composants shadcn / Base UI	mise en page responsive, mobile first
Animations	**GSAP**	apparitions et transitions
Base de données	**MongoDB** + **Mongoose 9**	33 collections (comptes, salons, réservations, avis, points…)
Authentification	**Auth.js v5**	connexion e-mail / mot de passe, Google, sessions sécurisées
Validation	**Zod**	contrôle de toutes les données reçues par l'API
Carte	**Leaflet** + OpenStreetMap	carte des salons

Domaine	Technologie	Rôle dans le projet
Graphiques	**Recharts**	statistiques du dashboard
E-mails	**nodemailer** (SMTP)	mot de passe oublié, rappels
Notifications	**Web Push**	rappels sur le téléphone
Tests	**node:test** + tsx	75 tests automatisés sur une base de test

3. L'architecture

Le code suit une **architecture propre (Clean Architecture)** : chaque domaine est un **module** indépendant dans `src/modules/<module>/`, découpé en 3 couches.

Couche	Contenu	Exemple
`domain`	règles métier pures, sans base de données, faciles à tester	calcul de l'acompte, verrou des créneaux
`application`	cas d'usage et validation des données (Zod)	« créer une réservation »
`infrastructure`	lecture et écriture dans MongoDB	requêtes Mongoose

Organisation des dossiers :

```
src/
  app/              pages et routes API (Next.js)
    (site)/         site public + espace client (/compte)
    (auth)/         connexion, inscription, mot de passe oublié
    dashboard/      espace du salon (gérant, employés)
    admin/          espace Super Admin
    api/            routes REST
  modules/          un dossier par domaine (identity, booking, reviews, loyalty…)
  components/       composants d'interface
  core/domain/      règles communes (erreurs, heure de Tunis, événements)
  infrastructure/   connexion MongoDB, sessions, outils HTTP
  i18n/             traductions français / arabe / anglais
public/sw.js        service worker (hors connexion, notifications)
tests/              tests automatisés
```

Les modules communiquent par événements. Exemple : quand le salon marque un rendez-vous « terminé », le module réservation publie `booking.completed`. Le module fidélité y réagit en créditant les points, et le module notifications prévient la cliente. Aucun module ne modifie le code d'un autre.

Tâches planifiées (toutes les 10 minutes) : rappels de rendez-vous, préavis et expiration des points, analyse des nouveaux avis.

4. La base de données

MongoDB stocke les données sous forme de documents. Les données qui vivent toujours avec leur parent sont **imbriquées** : les horaires et les prestations dans le salon, les lignes et le paiement dans la réservation, le compte fidélité dans l'utilisateur.

Collection	Contenu
`User`	comptes (rôle, mot de passe haché, points, favoris)

Collection	Contenu
`Salon`	salons (adresse, horaires, photos, prestations, plan)
`Employee`, `TimeOff`	équipe et congés
`Booking`	réservations (prestations, statut, acompte)
`SlotLock`	verrou anti double réservation (tranches de 5 min)
`Review`, `ReviewReport`	avis et signalements
`PointTransaction`, `Reward`, `Redemption`	fidélité
`Notification`, `PushSubscription`	notifications
`PlanPayment`	abonnements des salons
`AuditLog`	journal des actions sensibles

5. La sécurité

- Mots de passe hachés (bcrypt) ; compte bloqué 15 minutes après 5 échecs.
- Rôle vérifié sur **chaque page et chaque API**.
- Protection CSRF : une écriture venant d'un autre site est refusée.
- Toutes les données reçues sont validées (Zod) avant d'arriver en base.
- Les actions sensibles du Super Admin sont journalisées.

6. L'application mobile (PWA)

- **Manifest** : nom, icônes, mode plein écran, raccourcis. L'application s'installe sur l'écran d'accueil.
- **Mode hors connexion** : le service worker enregistre les pages consultées et précharge les pages publiques liées. Sans réseau, elles s'ouvrent depuis le téléphone ; sinon la page `/offline` liste les pages disponibles. Les pages personnelles sont effacées à la déconnexion.
- **Mobile first et responsive** : testé de 320 à 1440 px de large, en français, en arabe et en anglais, sans aucun débordement.

7. Lancer le projet

```
npm install
npm run db:seed        données de démonstration
npm run dev            développement : http://localhost:3000
npm run build && npm start      production (nécessaire pour le hors connexion)
npm test               75 tests automatisés
```

Sur la page de connexion, les boutons **« Accès démo par rôle »** connectent en un clic : Client, Gérant, Employé, Super Admin.

8. Les 5 gestions

#	Gestion	Responsable	IA associée
1	Comptes, sécurité et Super Admin	P1	Recommandation de salons
2	Dashboard du salon	P2	Prédiction d'affluence

#	Gestion	Responsable	IA associée
3	Recherche et réservation	P3	Assistante de réservation
4	Avis, modération et support	P4	Analyse des avis
5	Espace client, fidélité, abonnements et PWA	P5	Analyse beauté par photo

Un parcours complet relie les 5 gestions :

1. La cliente se connecte et voit des salons recommandés (gestion 1).
2. Elle réserve un créneau (gestion 3).
3. Le salon voit le rendez-vous dans son agenda et le marque terminé (gestion 2).
4. La cliente gagne des points (gestion 5) et laisse un avis vérifié, analysé par l'IA (gestion 4).
5. Le Super Admin suit l'activité et modère (gestion 1).

4 Gestion des avis, de la modération et du support (P4)

But : des avis **vérifiés et fiables**, et un moyen pour clientes et salons de signaler un problème.

Fonctionnalités

- **Avis vérifié** : possible seulement après un rendez-vous **terminé**, par la cliente concernée, une seule fois.
- Note de 1 à 5, commentaire et photo facultative.
- **Réponse du salon** à un avis, depuis `/dashboard/avis`.
- **Signalement** d'un avis par le salon ou par un utilisateur.
- **Modération** par le Super Admin (`/admin/moderation`) : il garde ou masque l'avis. La **décision finale est toujours humaine**.
- **Badge « Salon vérifié »** après validation du justificatif (gestion 1).
- **Support** : tickets ouverts par les clientes (`/compte/support`) et les salons (`/dashboard/support`), traités par le Super Admin (`/admin/support`).

IA — Analyse des avis

- **Entrée** : le texte et la note de chaque avis.
- **Méthode** :
  - **sentiment** calculé avec un lexique français, anglais et arabe (darija comprise) qui tient compte des négations et des emojis, puis croisé avec la note ;
  - **aspects** : accueil, résultat, attente, prix, propreté, conseil et ponctualité, chacun jugé positif ou négatif ;
  - **score de suspicion** de 0 à 100 : texte creux, note incohérente avec le texte, liens…
- **Sorties** :
  - sur la fiche du salon : la synthèse (répartition des sentiments, points forts, points à améliorer) ;
  - pour la modération : les avis suspects envoyés en file.
- **Repli** : si le texte ne contient aucun indice, le sentiment vient de la note seule. Les avis anciens sont analysés par la tâche planifiée.
- **Code** : `src/modules/reviews/domain/analysis.ts` · `POST /api/ai/reviews/analyze`.

Repères techniques

Modules	`reviews`, `support`
Pages	bloc avis de la fiche salon, `/dashboard/avis`, `/admin/moderation`, `/compte/support`, `/dashboard/support`, `/admin/support`
API	`/api/reviews/*`, `/api/ai/reviews/analyze`, `/api/support/*`, `/api/admin/support`
Collections	`Review`, `ReviewReport`, `SupportTicket`
Événements émis	`review.published` (la fidélité y réagit, gestion 5)
Tests	`loyalty-reviews`, `review-analysis`, `support`

Ma préparation pour la validation

Fichiers à connaître

- `src/modules/reviews/`, `src/modules/support/`
- Pages : `src/app/dashboard/avis`, `src/app/admin/moderation`, `src/app/admin/support`, `src/app/(site)/compte/support` · API : `src/app/api/reviews/*`, `src/app/api/support/*`
- Composants : `src/components/manage/reviews-manager.tsx`, `src/components/manage/support-*.tsx`
- IA : `src/modules/reviews/domain/analysis.ts` · tests : `tests/loyalty-reviews.test.ts`, `tests/review-analysis.test.ts`, `tests/support.test.ts`

Démo (3 minutes)

1. Connexion démo **Client** → un rendez-vous terminé → **laisser un avis** (Create) avec note et photo.
2. Fiche salon : l'avis apparaît (Read) avec la synthèse IA.
3. Connexion **Gérant** → `/dashboard/avis` : répondre à l'avis (Update), signaler un avis ; pagination (5 par page).
4. Connexion **Super Admin** → `/admin/moderation` : garder ou masquer un avis signalé.
5. Ticket de support ouvert par une cliente, réponse puis clôture par le Super Admin.

Montrer l'IA

- Où la montrer : synthèse sur la fiche salon et dans `/dashboard/avis`, avis suspects dans `/admin/moderation`.

Règles métier à savoir expliquer

- Avis possible **seulement après un rendez-vous terminé**, par la cliente concernée, une seule fois.
- La note moyenne du salon est recalculée à chaque avis publié ou masqué.
- La décision de modération est **toujours humaine** : l'IA signale, l'admin décide.

Questions probables du jury

- *Comment savez-vous qu'un avis est faux ?* On ne le sait pas avec certitude : l'IA donne un score de suspicion et l'envoie en modération ; un humain tranche.
- *Pourquoi pas un service d'IA externe ?* Gratuit, rapide, explicable, et les textes ne quittent pas le serveur.